TwinWare: Project Execution & Team Work Plan

1. PROJECT AT A GLANCE

Project Name: TwinWare **Full Title:** Design and Implementation of an Edge-Based Cyber-Physical Digital Twin Framework for Autonomous Warehouse AGVs using Raspberry Pi 5 and ESP32

What we are building: A miniature autonomous warehouse system where a physical Automated Guided Vehicle (AGV) is mirrored by a live Digital Twin. The system uses an ESP32 for real-time motor and sensor control, and a Raspberry Pi 5 as an edge node to process telemetry, host the backend, and maintain the Digital Twin state.

Why we are building it: To demonstrate edge-based computation and cyber-physical systems (CPS) in a practical, physical format, bridging the gap between low-level embedded hardware and high-level digital orchestration.

Main Technologies: * **Hardware:** ESP32, Raspberry Pi 5, DC Motors with Encoders, Sensors. * **Communication:** UART, MQTT. * **Software Stack:** C/C++ (ESP32), Python/FastAPI (Edge), SQLite, web-based dashboard (HTML/JS/CSS).

Final Demonstration: The AGV will complete a warehouse mission while dodging an obstacle. A web dashboard will simultaneously display the exact digital twin counterpart mirroring the AGV's movements and state in real-time.

Core Team: Two students: * Member A (Embedded & Robotics Lead) * Member B (Edge, Digital Twin & Software Lead)

Expected Final Outcome: A fully functional, synchronized physical-to-digital system capable of executing commands and displaying low-latency telemetry.

---

2. WHAT EXACTLY ARE WE BUILDING?

We are building a closed-loop Cyber-Physical System where a physical robot is continuously synchronized with a virtual model.

The Complete Flow:

```
Physical AGV
      ↓
Sensors + Motors
      ↓
```

```
ESP32
      ↓
UART
      ↓
Raspberry Pi 5
      ↓
MQTT + Backend + Database
      ↓
Digital Twin
      ↓
Web Dashboard
      ↓
Commands
      ↓
Raspberry Pi
      ↓
ESP32
      ↓
Physical AGV
```

The Digital Twin is **NOT merely a dashboard**. It is the central digital representation of the current state of the physical AGV and its environment. When the physical AGV moves, the twin moves. When a command is issued to the twin, it is translated and executed by the physical AGV.

---

3. FINAL DEMO — WHAT SHOULD WE BE ABLE TO SHOW?

This is our **Target Definition of Done**.

1. The AGV starts at Station A in the physical arena.
2. The Web Dashboard shows the AGV as “Online” at coordinates (0,0).
3. The Operator creates a mission to move to Station B.
4. The mission is sent to the AGV via the backend/MQTT.
5. The physical AGV begins moving.
6. The physical AGV continuously reports telemetry (speed, encoders, orientation).
7. The Digital Twin moves simultaneously on the dashboard.
8. The dashboard displays live position, speed, battery level, and sensor status.
9. An obstacle is unexpectedly introduced in front of the physical AGV.
10. The physical AGV detects it and halts.
11. The Digital Twin instantly reflects the “Obstacle Detected” status.
12. The obstacle is removed, and the mission continues.
13. The AGV reaches Station B.
14. The mission is marked “Completed” on the dashboard.
15. Historical telemetry and latency metrics can be viewed.

---

4. SYSTEM ARCHITECTURE

Physical Layer

- **Chassis & Locomotion:** Acrylic/3D printed chassis, DC geared motors, Wheels, Caster.
- **Sensors:** Wheel encoders (for odometry), IMU (for heading), Distance sensors (ultrasonic/ToF for obstacles).
- **Power:** Li-Po battery pack, Buck converters for Pi and ESP32.

Embedded Layer (ESP32)

Responsibilities: * Direct motor control via motor driver * PWM generation * Encoder reading via interrupts * Sensor reading (IMU, distance) * PID control for straight-line movement * Safety stop logic * UART communication with the edge node * Command execution

Edge Layer (Raspberry Pi 5)

Responsibilities: * UART communication (serial read/write) * MQTT Broker/Client * Telemetry processing and aggregation * Database hosting (SQLite) * Maintaining the core Digital Twin state * High-level navigation computation (waypoints) * Hosting the Backend API (FastAPI)

Digital Twin

Responsibilities: * Maintaining the virtual warehouse state * Syncing physical to virtual AGV * Providing live state data * Logging historical information * Flagging alerts

Dashboard

Responsibilities: * Live 2D monitoring interface * Mission creation and dispatch * Robot status visualization * Analytics and Diagnostics

---

5. COMPLETE COMPONENT LIST

A. Already Available

- [ ] ESP32 Development Board
- [ ] Wire jumpers, minor electronics

B. Mandatory Components

- [ ] Raspberry Pi 5
- [ ] Raspberry Pi official 27W power supply (for development)
- [ ] Active cooling (fan/heatsink) for Pi 5

- [ ] MicroSD card (32GB+ Class 10)
- [ ] Robot chassis (2-wheel drive)
- [ ] DC geared motors (x2)
- [ ] L298N or TB6612FNG Motor driver
- [ ] Wheels (x2)
- [ ] Caster wheel (x1)
- [ ] Wheel encoders (x2)
- [ ] IMU (MPU6050)
- [ ] Ultrasonic (HC-SR04) or ToF (VL53L0X) sensors
- [ ] Li-Po Battery (e.g., 3S 11.1V)
- [ ] Battery voltage monitor
- [ ] LM2596 Buck converter (for 5V step-down)
- [ ] Wiring (Jumper wires, thick gauge wire for power)
- [ ] Connectors (XT60, header pins)
- [ ] Main power switch
- [ ] Emergency stop button
- [ ] Mounting hardware (standoffs, screws)

C. Optional Components (DO NOT MAKE MANDATORY)

- [ ] Raspberry Pi Camera
- [ ] AprilTags for localization
- [ ] RFID reader/tags
- [ ] LiDAR
- [ ] Load cell
- [ ] OLED Display
- [ ] Additional sensors (e.g., line followers)

---

6. COMPONENT SELECTION

Component	Recommended Choice	Why	Alternative	Mandatory?
Edge Node	Raspberry Pi 5	Edge computing power, runs Linux, handles backend + MQTT + database easily.	Raspberry Pi 4	Yes
Controller	ESP32	Fast, lots of GPIO, built-in interrupts, real-time response for PID.	Arduino Mega	Yes
Motor Driver	TB6612FNG	Smaller, more	L298N	Yes

Component	Recommended Choice	Why	Alternative	Mandatory?
		efficient than L298N, lower voltage drop.		
IMU	MPU6050	Cheap, well-documented, sufficient for basic heading.	BNO085 (expensive)	Yes
Distance	HC-SR04	Extremely cheap, good enough for a basic stop trigger.	VL53L0X (ToF)	Yes
Database	SQLite	Zero-config, lightweight, perfect for this scale.	InfluxDB	Yes

7. HARDWARE BUILD PLAN

Step 1: Build Chassis

- **What to do:** Assemble the base plates.
- **Required:** Chassis kit, screwdrivers.
- **Who:** Both
- **Expected Result:** A stable platform to mount electronics.

Step 2: Mount Motors & Encoders

- **What to do:** Secure DC motors and attach encoder disks.
- **Required:** Motors, encoders, brackets.
- **Who:** Member A
- **Expected Result:** Motors are physically attached and encoders can rotate.

Step 3: Install Wheels & Caster

- **What to do:** Attach the main wheels and the balancing caster.
- **Who:** Member A
- **Expected Result:** Chassis can roll smoothly on the floor.

Step 4: Mount ESP32 & Motor Driver

- **What to do:** Secure the embedded electronics on the lower deck.

- **Who:** Member A
- **Expected Result:** Electronics are secured and won't short circuit on the frame.

Step 5: Mount Raspberry Pi

- **What to do:** Secure Pi 5 and cooling fan on the upper deck.
- **Who:** Member B
- **Expected Result:** Pi 5 is accessible and cooled.

Step 6: Mount Sensors

- **What to do:** Place the distance sensor at the front, IMU near the center of rotation.
- **Who:** Member A
- **Expected Result:** Clear line of sight for distance sensor.

Step 7: Power Distribution

- **What to do:** Install battery, switch, buck converters. Tune buck converter output to EXACTLY 5V before connecting to Pi/ESP32.
- **Who:** Both
- **What can go wrong:** Reversing polarity or sending 11.1V to the 5V rail will destroy the electronics.

Step 8: Complete Power Wiring

- **What to do:** Connect battery -> Switch -> Buck Converter -> Pi/ESP32.
- **Who:** Both
- **Expected Result:** Turn switch on, LEDs on buck converter light up.

Step 9: Signal Wiring

- **What to do:** Connect ESP32 to motor driver, encoders, IMU, sensors. Connect ESP32 to Pi via UART.
- **Who:** Member A
- **Expected Result:** All data connections are made.

Step 10: Perform Power-on Test

- **What to do:** Turn on power, verify LEDs, check for smoke/heat.
- **Who:** Both
- **Expected Result:** System boots up without hardware failures.

---

8. TWO-PERSON WORK DIVISION

MEMBER A — Embedded & Robotics Lead

Responsible for: * ESP32 firmware (C/C++) * Motor control & PWM * Encoder reading & interrupts * IMU integration * Distance and other sensor readings * PID tuning for straight-line movement * UART protocol formatting and transmission * Robot navigation (low-level logic) * Hardware integration & physical wiring

MEMBER B — Edge, Digital Twin & Software Lead

Responsible for: * Raspberry Pi 5 Linux OS setup * MQTT broker configuration (Mosquitto) * Backend API (FastAPI, Python) * Database management (SQLite) * Digital Twin state management logic * Web Dashboard development (HTML/JS/CSS) * Telemetry processing and aggregation * Data visualization and UI mapping

Shared Responsibilities

Both members jointly handle: * System architecture planning * Hardware integration * End-to-end testing * Debugging communication between Edge and Embedded * Documentation * GitHub management (code reviews, merges) * Demo preparation * Professor updates * Final presentation * Research measurements and performance tracking

---

9. RESPONSIBILITY MATRIX

Task	Member A	Member B	Shared	Dependency
Hardware Procurement			X	Phase 0
Chassis Assembly			X	Procurement
Motor/Power Wiring	X			Assembly
ESP32 Firmware Base	X			IDE setup
Encoder Integration	X			Firmware Base
IMU & Distance	X			Wiring
PID Control	X			Encoders
Raspberry Pi Setup		X		Pi procured
MQTT Broker Setup		X		Pi Setup
UART Protocol Spec			X	Pi + ESP32
UART Implement (ESP)	X			Protocol Spec
UART Implement (Pi)		X		Protocol Spec
Backend & DB		X		Pi Setup
Digital Twin Logic		X		Backend
Navigation Logic	X			PID
Dashboard UI		X		Digital Twin

Task	Member A	Member B	Shared	Dependency
Mission Management		X		Dashboard UI
Integration			X	All Modules
Testing & Debugging			X	Integration
Presentation/Docs			X	Testing

---

10. DEVELOPMENT PHASES

PHASE 0 — Planning

- Finalize architecture and component list.
- Divide responsibilities clearly.
- Create GitHub repository.

PHASE 1 — Hardware Setup

- Procure components.
- Build chassis, test motors with 12V direct, test power voltages.
- Mount and test all sensors physically.

PHASE 2 — ESP32 Firmware

- Setup GPIO, PWM, and basic motor control.
- Implement encoder reading via hardware interrupts.
- Read IMU and distance sensors.
- Battery monitoring.
- Implement PID loop for speed control.
- Safety stop logic.
- UART base setup.

PHASE 3 — Raspberry Pi Setup

- Install Raspberry Pi OS.
- Setup Python, Git environment.
- Install and test MQTT Broker (Mosquitto).
- Initialize FastAPI project and SQLite database.

PHASE 4 — Communication

- Define UART protocol (JSON structure).
- Send ESP32 telemetry to Pi, read commands from Pi.
- Define MQTT topics (`agv/telemetry`, `agv/commands`).
- Implement error handling and heartbeat signals.

PHASE 5 — Robot Navigation

- Motor calibration (ticks to meters).
- Encoder-based movement and odometry.
- Basic waypoint navigation (point-to-point).
- Obstacle detection -> automatic stop/recovery.

PHASE 6 — Digital Twin

- Model the warehouse layout digitally.
- Create virtual AGV state representation.
- Synchronize physical position and status via MQTT.
- Log sensor states and mission status.

PHASE 7 — Dashboard

- Build dashboard overview screen.
- Display AGV details and alerts.
- Implement mission control (Go to Station B).
- View analytics and diagnostics.

PHASE 8 — Full Integration

- Connect everything:
  - `AGV -> ESP32 -> UART -> Raspberry Pi -> MQTT -> Digital Twin -> Dashboard`
  - `Dashboard -> Backend -> MQTT -> Raspberry Pi -> UART -> ESP32 -> AGV`

PHASE 9 — Testing

- Test movement accuracy, sensor reliability.
- Test communication robustness.
- Test Navigation, Digital Twin synchronization.
- Test Dashboard UI responsiveness.
- Failure recovery and Emergency stop.

PHASE 10 — Final Demonstration

- Prepare the complete warehouse demo scenario.

---

11. PHASE-BY-PHASE TASK CHECKLIST

(Example checklist for Phase 4)

PHASE 4: Communication

- **Objective:** Establish reliable bi-directional data flow between the ESP32 and the Pi.

- **Tasks:** Agree on JSON format, implement parsing on ESP32, implement reading/writing on Pi via Python, publish to MQTT.
- **Member A:** Write `send_telemetry()` and `parse_command()` over Serial on ESP32.
- **Member B:** Write Python scripts `serial_to_mqtt.py` and `mqtt_to_serial.py`.
- **Dependencies:** ESP32 basic code, Pi OS, MQTT broker.
- **Deliverable:** The Pi prints live sensor data; ESP32 turns on motors when an MQTT message is published.
- **Completion Criteria:** A command sent via MQTT physically moves the robot, and telemetry immediately reflects the movement.
- **GitHub Commit:** `feature/uart-mqtt-bridge`
- **Report to Professor:** "We have achieved full communication from the edge node to the embedded controller and into the MQTT broker."

---

12. WEEK-BY-WEEK PLAN (16 WEEKS)

Week	Goal
Week 1	Planning + architecture + ordering components
Week 2	Hardware Procurement + Chassis Assembly + Power Testing
Week 3	ESP32 Basic Motor Control & Sensor Integration
Week 4	Encoders + PID tuning
Week 5	Raspberry Pi OS + Mosquitto MQTT Setup
Week 6	UART Bridge (ESP32 <-> Pi)
Week 7	Odometry and Basic Navigation (Dead reckoning)
Week 8	Backend API & SQLite Database
Week 9	Navigation refinement (waypoint following)
Week 10	Digital Twin State Management Logic
Week 11	Web Dashboard Interface (2D map)
Week 12	Full System Integration
Week 13	Testing: Obstacle detection, edge cases
Week 14	Optimization and Performance Measurement
Week 15	Demo Preparation & Research Measurement extraction
Week 16	Final Presentation & Documentation hand-in

---

13. MVP VS ADVANCED FEATURES

> **CRITICAL RULE: Never delay the MVP because an advanced feature is incomplete.**

MVP — MUST WORK

- AGV physical movement
- ESP32 motor control + encoders

- Basic obstacle detection (stop if blocked)
- UART communication
- Raspberry Pi processing
- MQTT broker functioning
- Basic Digital Twin tracking X, Y, Theta
- Dashboard showing live status
- Mission execution from Dashboard
- Emergency stop mechanism

LEVEL 2 — SHOULD HAVE

- Better localization and Odometry tuning
- PID parameter optimization
- Historical data charts on Dashboard
- Analytics and latency measurement logs
- Better Digital Twin 2D visualization

LEVEL 3 — OPTIONAL / FUTURE WORK

- LiDAR / SLAM integration
- Computer vision / AprilTags
- Predictive maintenance AI
- 3D Cloud Digital Twin
- ROS2 integration
- Fleet management (Multi-AGV)

---

14. SOFTWARE STACK

ESP32

- **Language:** C/C++
- **Framework:** Arduino Framework or ESP-IDF

Edge (Raspberry Pi 5)

- **OS:** Raspberry Pi OS
- **Language:** Python 3
- **Communication:** `pyserial`, Eclipse Mosquitto (MQTT)

Backend

- **Framework:** FastAPI (Python) - lightweight and fast API handling.

Database

- **Primary:** SQLite - Native, file-based, perfect for telemetry storage at this scale. No extra background services needed.

- **Alternative:** InfluxDB ONLY if time-series analytics is strictly required for advanced features.

Dashboard

- **Frontend:** Vanilla HTML/JS/CSS or a very simple Vue.js setup. Do not over-engineer with React/Webpack for the initial version.
- **Visualization:** 2D Canvas API or SVG for the Digital Twin map. Do not use 3D (Three.js/Unity) until MVP is fully complete.

---

15. DATA FLOW

Telemetry Flow (Upward):

```
Encoder / IMU / Distance / Battery
      ↓
    ESP32
      ↓
    UART
      ↓
Raspberry Pi (Python Worker)
      ↓
    MQTT (topic: agv/telemetry)
      ↓
Telemetry Processor (FastAPI)
      ↓
Database + Digital Twin State
      ↓
Dashboard (via WebSocket/Polling)
```

Command Flow (Downward):

```
Dashboard (User issues Mission)
     ↓
FastAPI Backend
     ↓
MQTT (topic: agv/commands)
     ↓
Raspberry Pi (Python Worker)
     ↓
    UART
     ↓
    ESP32
     ↓
Motor Controller Logic
```

---

16. BASIC DATA FORMAT

Keep JSON structures simple and flat.

Telemetry JSON (ESP32 -> Pi):

```
{
  "agv_id": "AGV01",
  "timestamp": 1690000000,
  "x": 1.25,
  "y": 2.50,
  "theta": 90.0,
  "speed": 0.4,
  "left_rpm": 60,
  "right_rpm": 60,
  "battery": 11.4,
  "obstacle": false
}
```

Command JSON (Pi -> ESP32):

```
{
  "command": "MOVE",
  "linear_velocity": 0.2,
  "angular_velocity": 0.0
}
```

---

17. GITHUB STRUCTURE

Keep the repository logically separated for both team members.

```
TwinWare/
│
├── firmware/
│   └── esp32/                # C++ code, PlatformIO/Arduino files
│
├── raspberry-pi/
│   ├── uart/                 # pyserial <-> MQTT bridging scripts
│   ├── mqtt/                 # MQTT configuration files
│   ├── navigation/           # High level waypoint generation
│   ├── digital-twin/         # Core twin logic, state syncing
│   ├── backend/              # FastAPI server
│   └── database/             # SQLite setup and models
│
├── dashboard/                # HTML, JS, CSS frontend files
│
├── hardware/
│   ├── wiring/               # Wiring diagrams
│   ├── schematics/           # Pinouts
│   └── bom/                  # Bill of Materials
```

```
│
├── tests/                    # Unit testing scripts
│
├── docs/                     # Weekly reports, this document
│
└── README.md
```

---

18. GIT WORKFLOW FOR TWO PEOPLE

- **`main` branch:** ALWAYS contains working, testable code. Never push broken code here.
- **`develop` branch:** Integration branch for testing components together.
- **Feature Branches:** Use descriptive names.
  - e.g., `feature/esp32-motor-control`
  - e.g., `feature/uart`
  - e.g., `feature/digital-twin`

Rules: 1. Work on your respective feature branch. 2. Create Pull Requests into `develop`. 3. Review together to avoid overwriting each other's work. 4. Merge `develop` to `main` when a milestone is reached.

---

19. DAILY / WEEKLY REPORTING SYSTEM

Do not create complicated project-management bureaucracy. Keep it simple.

Daily Log (On Discord/Notes): * Today I worked on: * Completed: * Problems: * Next task: * Commit/PR:

Weekly Report: * Week: # *** Completed work: * Pending work: * Problems / Hardware / Software issues: * Next week's goal: * Integration status: * Demo status:**

---

20. PROFESSOR REPORTING FORMAT

Keep it to a quick update, covering these 6 points:

1. **What we planned:**
2. **What we completed:**
3. **What is currently working:**
4. **What is blocked:**
5. **What we will do next:**
6. **What we need from the professor:**

During major milestones, physically demonstrate what is working.

---

21. INTEGRATION MILESTONES

- **MILESTONE 1:** ESP32 controls motors successfully.
- **MILESTONE 2:** ESP32 reads sensors accurately.
- **MILESTONE 3:** ESP32 ↔ Raspberry Pi UART communication works.
- **MILESTONE 4:** MQTT telemetry publishing and subscribing works.
- **MILESTONE 5:** AGV performs basic physical navigation.
- **MILESTONE 6:** Digital Twin Backend accurately tracks AGV state.
- **MILESTONE 7:** Dashboard controls and monitors the AGV in real-time.
- **MILESTONE 8:** Complete warehouse mission runs successfully end-to-end.
- **MILESTONE 9:** Final testing and performance measurements recorded.

---

22. TESTING — KEEP IT PRACTICAL

Hardware: * **Motor test:** Hook motors directly to verify they work. * **Sensor test:** Wave hand, verify readings in serial monitor. * **Power test:** Check voltages with multimeter. * **Encoder test:** Verify tick counts when spinning wheels manually.

Firmware: * **PWM:** Sweep values. * **PID:** Test response to disturbances. * **UART:** Send strings, verify reception. * **Safety:** Test emergency stop reaction time.

Communication: * **Telemetry:** Verify correct JSON format reaches MQTT. * **Commands:** Verify correct commands reach ESP32. * **Disconnect/reconnect:** Unplug UART, ensure safe shutdown.

Navigation: * **Straight movement:** Command 1 meter, measure error. * **Turning:** Command 90 degrees, measure error. * **Obstacle:** Place object, ensure stop.

Digital Twin & Dashboard: * **Position synchronization:** Verify digital movement matches physical. * **State synchronization:** Verify battery/sensor data updates. * **Command synchronization:** Verify UI buttons trigger actions. * **Mission / Alerts:** Test complete flow from UI.

End-to-End: * **Complete warehouse mission test.**

---

23. RESEARCH / PERFORMANCE METRICS

Only measure what is genuinely useful. Do NOT invent results.

1. **Digital Twin synchronization latency:** (Time from physical move to UI update).
2. **Command latency:** (Time from UI click to physical movement).
3. **Localization error:** (Difference between Digital X,Y and physical tape-measured X,Y).
4. **Obstacle response time:** (Time from detection to full motor stop).
5. **MQTT packet loss:** (Messages dropped per hour).
6. **AGV mission success rate:** (% of missions completed without manual intervention).
7. **Raspberry Pi CPU/memory usage:** (Resource consumption during full operation).

---

24. COMMON FAILURE SCENARIOS

Problem: AGV does not move * Check: Battery charge, motor driver power, PWM output from ESP32, physical wiring.

Problem: Encoder readings incorrect * Check: GPIO pin assignment, interrupt setup, encoder wiring, CPR configuration.

Problem: Raspberry Pi doesn't receive telemetry * Check: UART TX/RX swapped, baud rate mismatch, serial device permissions (`/dev/ttyS0`), ESP32 actually sending data.

Problem: MQTT doesn't update * Check: Mosquitto broker running, correct topic, subscriber connected, network issues.

Problem: Digital Twin is out of sync * Check: Timestamp discrepancies, MQTT delays, telemetry processor lag, state update logic in backend.

---

25. FINAL DEFINITION OF DONE

The project is considered complete only when:

- ☐ Physical AGV operates
- ☐ Motors are controlled by ESP32
- ☐ Encoders work
- ☐ Sensors work
- ☐ Obstacle detection works
- ☐ ESP32 ↔ Raspberry Pi communication works
- ☐ MQTT works
- ☐ Raspberry Pi processes telemetry
- ☐ Digital Twin updates from physical AGV
- ☐ Dashboard displays live state
- ☐ Dashboard can issue at least basic commands
- ☐ AGV completes a predefined mission
- ☐ Emergency stop works
- ☐ Data is logged
- ☐ Performance metrics are measured
- ☐ Complete demonstration works

---

26. WHAT NOT TO BUILD

Explicitly do **NOT** waste time on: * Multi-AGV fleet management * Full industrial warehouse simulation * Cloud deployment (Keep it local to the Pi) * Complex 3D graphics * Advanced AI / Computer Vision * SLAM (Simultaneous Localization and Mapping) before basic navigation works * ROS2 (unless strictly required by the professor) * Over-engineered authentication systems * Excessive dashboard features * Any feature that does not contribute to the core Digital Twin objective

---

27. FINAL PROJECT ROADMAP — ONE PAGE

```
  PLANNING & ARCHITECTURE
            ↓
    BUY COMPONENTS
            ↓
BUILD AGV CHASSIS & WIRING
            ↓
ESP32: SENSORS + MOTORS + PID
            ↓
PI: UART + MQTT SETUP
            ↓
COMMUNICATION BRIDGE
            ↓
NAVIGATION & ODOMETRY
            ↓
BACKEND & DIGITAL TWIN LOGIC
            ↓
    WEB DASHBOARD
            ↓
  FULL SYSTEM INTEGRATION
            ↓
  TESTING & METRICS LOGGING
            ↓
FINAL DEMO & PRESENTATION
```

Development and Deployment Guide

This guide outlines our exact workflow. We will complete **Phase 1** entirely on your local laptop using simulation tools. Only when everything is working perfectly will we proceed to **Phase 2** (the physical Raspberry Pi).

Phase 1: Local Development (Laptop)

Goal: Build and test the complete software ecosystem (Backend, Frontend, and Embedded stubs) locally before touching the hardware.

✅ Completed So Far

- Scaffolded the Edge Node backend (`raspberry-pi/backend`).
- Created the UART-to-MQTT python bridge with a built-in **simulation mode** (`raspberry-pi/uart`).
- Containerized the entire edge stack using Docker and Docker Compose.

⏳ Exact Next Steps (To Do Now)

We will execute these steps one by one on your laptop:

1. **Build the Web Dashboard (Frontend)**
   - Create a `dashboard/` folder.
   - Build the HTML/JS/CSS interface to display the AGV's live position (Digital Twin), battery status, and provide a control panel for missions.
2. **Enhance the FastAPI Backend**
   - Add **WebSocket** support to `backend/main.py`. The dashboard needs WebSockets to receive instant telemetry updates without constantly refreshing the page.
3. **Scaffold the ESP32 Firmware**
   - Create a `firmware/esp32/` folder.
   - Set up the basic C++ files (PlatformIO/Arduino) that will later be uploaded to the ESP32 to handle motors, encoders, and UART communication.
4. **Local End-to-End Simulation Testing**
   - Run `docker compose up -d` on your laptop.
   - The simulation mode will generate fake AGV telemetry.
   - We will open the Dashboard in your laptop's browser and ensure the Digital Twin moves based on the simulated data.

Phase 2: Raspberry Pi Deployment

Goal: Move the completed, tested software onto the physical hardware.

Once Phase 1 is fully tested, follow these exact steps to deploy to the Raspberry Pi:

Step 1: Prepare the Pi

1. Flash **Ubuntu Server OS** onto the Raspberry Pi's SD card.
2. Connect the Pi to your Wi-Fi network.
3. From your laptop, **SSH** into the Pi:

   ```
   ssh username@<raspberry-pi-ip>
   ```

4. Install Docker on the Pi:

   ```
   curl -fsSL https://get.docker.com -o get-docker.sh
   sudo sh get-docker.sh
   sudo usermod -aG docker $USER
   ```

   (Log out and log back in for docker permissions to apply).

Step 2: Transfer the Code

You can use `scp` (Secure Copy) to send the required folders from your laptop to the Pi, or push everything to GitHub and clone it on the Pi.

Using `scp` from your laptop terminal:

```
scp -r "c:\Users\santh\OneDrive - vit.ac.in\Desktop\XXX\CyberTwin-
        AGV\raspberry-pi" username@<raspberry-pi-ip>:~/
scp -r "c:\Users\santh\OneDrive - vit.ac.in\Desktop\XXX\CyberTwin-
        AGV\dashboard" username@<raspberry-pi-ip>:~/
```

Step 3: Connect Hardware & Run

1. SSH into the Pi again.
2. Plug the ESP32 into the Pi via USB.
3. Check what port the ESP32 was assigned:

   ```
   ls /dev/ttyUSB*
   ```

   (If it says `/dev/ttyACM0` instead, update the `SERIAL_PORT` variable inside your `docker-compose.yml` file).
4. Navigate into the folder:

   ```
   cd ~/raspberry-pi
   ```
5. Start the edge servers:

   ```
   docker compose up -d --build
   ```
6. **Success!** You can now open a browser on your laptop and go to `http://<raspberry-pi-ip>:8000` to see the live Digital Twin communicating with the physical robot! # Project Status & Hardware Deployment Guide

✅ What Has Been Completed (Phase 1)

We have successfully completed all local software scaffolding and simulation setup for the CyberTwin-AGV project. Here is the current state of the repository:

1. **Edge Backend Server (`raspberry-pi/backend/`)**
   - Built a FastAPI server to manage the Digital Twin state.
   - Connected an SQLite database with SQLAlchemy to store telemetry records.
   - Integrated WebSockets and a background MQTT client (`paho-mqtt`) to broadcast real-time telemetry instantly without page reloads.
   - Configured FastAPI to serve the Frontend Dashboard directly, so everything runs from a single port (8000).
2. **MQTT Broker & Bridge (`raspberry-pi/uart/` & `mqtt/`)**
   - Added an Eclipse Mosquitto MQTT broker to the stack.
   - Developed `serial_mqtt_bridge.py`: a Python script that bridges the physical UART connection (to the ESP32) to the MQTT broker. It includes a built-in simulation mode that generates mock odometry data when the serial port isn't available.
3. **Digital Twin Web Dashboard (`dashboard/`)**
   - Designed a premium, glassmorphism-style UI using pure HTML/JS/CSS.
   - Features real-time telemetry gauges (Battery, Speed, Obstacles).

- Features a live 2D HTML5 `<canvas>` rendering of the AGV's exact position and orientation based on live data.
- Features a control pad that sends REST API commands to the backend, which are then pushed via MQTT down to the firmware.

4. **ESP32 Firmware Scaffold (`firmware/esp32/`)**
   - Set up a standard PlatformIO project.
   - Wrote `main.cpp` with UART initialization (115200 baud).
   - Implemented JSON parsing (via `ArduinoJson`) to read incoming target velocities, and JSON serialization to transmit odometry/telemetry back to the Pi.
   - Implemented a temporary simulation physics loop on the ESP32 so it can be tested independently of physical motors.
5. **Dockerization (`raspberry-pi/`)**
   - Created a complete `docker-compose.yml` and `Dockerfile` that packages the backend, the bridge, the MQTT broker, and mounts the dashboard into a unified, reproducible stack.

🚀 Exact Next Steps: Phase 2 (Hardware Deployment)

You are now ready to deploy this software stack onto your physical hardware. Follow these exact steps:

Step 1: Flash the ESP32 Firmware

1. Open the `firmware/esp32/` folder in Visual Studio Code (ensure you have the **PlatformIO** extension installed).
2. Connect your ESP32 to your laptop via USB.
3. Click the PlatformIO "Upload" button (the right-arrow icon in the bottom taskbar). This will compile the C++ code and flash it to the microcontroller.
4. *Note: For the real physical robot, you will eventually replace the simulation logic in `main.cpp` with actual PWM `analogWrite()` commands to your motor drivers and interrupts for your encoders.*

Step 2: Prepare the Raspberry Pi

1. Ensure the Raspberry Pi is running Ubuntu Server or Raspberry Pi OS and is connected to your Wi-Fi network.

2. SSH into your Raspberry Pi from your laptop:

   ```
   ssh <username>@<raspberry-pi-ip>
   ```

3. Install Docker on the Pi (if not already installed):

   ```
   curl -fsSL https://get.docker.com -o get-docker.sh
   sudo sh get-docker.sh
   sudo usermod -aG docker $USER
   ```

   (Log out and log back in for docker permissions to apply).

Step 3: Transfer the Code to the Pi

Since we need the `raspberry-pi` folder and the `dashboard` folder, transfer them using Secure Copy (`scp`) from your laptop's terminal:

```
scp -r "c:\Users\santh\OneDrive - vit.ac.in\Desktop\XXX\CyberTwin-
        AGV\raspberry-pi" <username>@<raspberry-pi-ip>:~/
scp -r "c:\Users\santh\OneDrive - vit.ac.in\Desktop\XXX\CyberTwin-
        AGV\dashboard" <username>@<raspberry-pi-ip>:~/
```

(Alternatively, push the code to a GitHub repository and `git clone` it on the Pi).

Step 4: Hardware Connections

1. Plug the flashed ESP32 into the Raspberry Pi using a USB cable.
2. On the Pi (via SSH), check which serial port was assigned:

   ```
   ls /dev/ttyUSB*
   # or
   ls /dev/ttyACM*
   ```

3. Open `~/raspberry-pi/docker-compose.yml` on the Pi (using `nano` or `vim`).
4. Update the `SERIAL_PORT` environment variable under the `bridge` service to match the port you found (e.g., `/dev/ttyUSB0`).
5. Also, ensure the `devices` mapping in `docker-compose.yml` is uncommented and points to the correct port so the Docker container can access the physical USB port.

Step 5: Launch the Stack on the Pi

1. On the Pi (via SSH), navigate to the folder:

   ```
   cd ~/raspberry-pi
   ```

2. Build and start the edge servers:

   ```
   docker compose up -d --build
   ```

3. **Verify**: Open a web browser on your laptop and go to:

   ```
   http://<raspberry-pi-ip>:8000
   ```

   You should see the Digital Twin Dashboard. Use the D-Pad to send commands—these will travel via WebSockets -> FastAPI -> MQTT -> UART Bridge -> ESP32!